TrendSurfer Pro 🚀 — Versione Minimal Sweep RR (Pine Script v6)

Motore Sweep RR con logica invariata (Breakeven, Trailing Stop, un trade alla volta). Novità di questa versione: SL Buffer e Trailing Step espressi in % del prezzo (validi su qualsiasi coin, nessun ricalcolo), linee Entry/SL/TP opzionali e pallino al prezzo esatto di chiusura.

▲ Triangolo verde	Ingresso LONG — apertura trade su Liquidity Sweep rialzista (tutti i filtri mantenuti)
▼ Triangolo rosso	Ingresso SHORT — apertura trade su Liquidity Sweep ribassista (tutti i filtri mantenuti)
× Crocetta verde	Uscita dei LONG — TP / SL / Breakeven / Trailing Stop colpito
× Crocetta rossa	Uscita degli SHORT — TP / SL / Breakeven / Trailing Stop colpito
Linee (opzionali)	Entry grigia tratteggiata · SL rossa (si sposta con BE/Trailing) · TP verde
● Pallino (opzionale)	Prezzo esatto di chiusura: sul TP = take · sulla SL = stop · in mezzo = trailing/breakeven

```
//@version=6
indicator(title = 'TrendSurfer Pro Sweep RR Minimal 🚀', overlay = true, max_lines_count = 500, max_labels_count = 500)

// ═══════════════════════════════════════════════════════
// TRENDSURFER PRO - VERSIONE MINIMAL (SWEEP RR SYSTEM)
// Stessa strategia e stessi filtri dell'originale, cambia SOLO la
// visualizzazione. Sul grafico compaiono:
//   ▲ verde  = punto d'ingresso LONG  (apertura trade su Liquidity Sweep rialzista)
//   ▼ rosso  = punto d'ingresso SHORT (apertura trade su Liquidity Sweep ribassista)
//   × verde  = punto d'uscita dei LONG  (TP / SL / Breakeven / Trailing Stop colpito)
//   × rossa  = punto d'uscita degli SHORT (TP / SL / Breakeven / Trailing Stop colpito)
// Opzionali (attivi di default, disattivabili nelle impostazioni):
//   linee Entry (grigia) / SL (rossa) / TP (verde) di ogni trade
//   ● pallino al prezzo ESATTO di chiusura del trade: sul TP = take,
//     sulla linea SL = stop, in mezzo = trailing stop o breakeven
// SL Buffer e Trailing Step sono espressi in % DEL PREZZO: gli stessi
// valori vanno bene su qualsiasi coin e non vanno mai ricalcolati.
// La logica del motore di trade Sweep RR è invariata: SL oltre la wick
// + buffer, TP sul precedente swing (o RR fisso in Custom RR Mode),
// Breakeven, Trailing Stop, un solo trade alla volta (salvo Overlap).
// Tutti i filtri dei segnali sweep sono mantenuti: filtri orari,
// EMA750/VIDYA/VWAP sugli sweep, ATR Volatility Score, ATR Mode.
// ═══════════════════════════════════════════════════════

// === TIMEZONE SETTINGS ===
useUTC = input.bool(true, "Use UTC Timezone ⏰", group = "Timezone")
utcOffset = input.int(2, "UTC Offset (hours)", minval = -12, maxval = 14, group = "Timezone", tooltip = "Set your timezone offset from UTC. Auto-adjusts when changing assets - Imposta l'offset del fuso orario rispetto all'UTC.")

// === ATR VOLATILITY SCORE ===
atrVolPeriod = input.int(14, "ATR Period (Volatility Score)", minval = 1, group = "ATR Volatility Score 📊")
atrVolMAPeriod = input.int(5, "ATR MA Period (score calibration)", minval = 1, group = "ATR Volatility Score 📊", tooltip = "Moving average of the ATR used as a reference. ATR > MA = high volatility, ATR < MA = low volatility")
atrVolSensitivity = input.float(2.0, "Sensitivity (1-5)", minval = 0.5, maxval = 5.0, step = 0.5, group = "ATR Volatility Score 📊", tooltip = "Amplifies or reduces the score's response. Higher values = more responsive")
enableVolScoreFilter = input.bool(false, "Enable Volatility Score Master Filter 🚦", group = "ATR Volatility Score 📊", tooltip = "Show signals only if ATR Volatility Score >= minimum threshold")
volScoreMinThreshold = input.int(80, "Min Volatility Score (1-100)", minval = 1, maxval = 100, group = "ATR Volatility Score 📊")

atrVol = ta.atr(atrVolPeriod)
atrVolMAAdapted =
     timeframe.in_seconds() <= 60 ? atrVolMAPeriod * 5 :
     timeframe.in_seconds() <= 120 ? atrVolMAPeriod * 3 :
     timeframe.in_seconds() <= 180 ? atrVolMAPeriod * 2 :
     timeframe.in_seconds() <= 300 ? atrVolMAPeriod :
     timeframe.in_seconds() <= 900 ? math.round(atrVolMAPeriod / 2) :
     timeframe.in_seconds() <= 1800 ? math.round(atrVolMAPeriod / 3) :
     timeframe.in_seconds() <= 3600 ? math.round(atrVolMAPeriod / 5) :
     timeframe.in_seconds() <= 14400 ? math.round(atrVolMAPeriod * 3) :
     atrVolMAPeriod
atrVolMA = ta.sma(atrVol, math.max(2, atrVolMAAdapted))
atrVolDev = atrVolMA > 0 ? (atrVol / atrVolMA - 1.0) * 100.0 * atrVolSensitivity : 0.0
atrVolScore = math.max(1, math.min(100, math.round(50 + atrVolDev)))

// === ATR MODE ===
enableATRMode = input.bool(false, title = 'Enable ATR Mode 📊', group = "ATR Bands Filter", tooltip = "When enabled, signals are filtered: only signals within the ATR channel around EMA750 are shown.")
atrMultiplier = input.float(5, title = 'ATR Channel Multiplier (1-100)', minval = 1, maxval = 100, step = 0.5, group = "ATR Bands Filter")

// === LIQUIDITY SWEEP (INGRESSI) ===
enableLiqSweep = input.bool(true, "Enable Liquidity Sweeps ❌", group = "Liquidity Sweep 💧")
sweepLookback_TS = input.int(1, "Sweep Lookback (re-entry candles)", group = "Liquidity Sweep 💧", minval = 1, maxval = 10, tooltip = "Maximum number of candles after a spike beyond a swing level to confirm price has re-entered the range.")
swingLenSweep_TS = input.int(5, "Swing Length — Sweep", group = "Liquidity Sweep 💧", minval = 1, maxval = 50, tooltip = "Number of candles left & right used to identify swing highs/lows for Liquidity Sweep detection.")
sweepEMAFilter_TS = input.bool(true, "Apply EMA750 Filter to Sweeps", group = "Liquidity Sweep 💧", tooltip = "Bullish sweeps only above EMA750, bearish only below")
sweepVIDYAFilter_TS = input.bool(false, "Apply VIDYA Filter to Sweeps", group = "Liquidity Sweep 💧", tooltip = "Bullish sweeps only above VIDYA, bearish only below")
sweepVWAPFilter_TS = input.bool(false, "Apply VWAP Filter to Sweeps", group = "Liquidity Sweep 💧")

// === SWEEP RR SYSTEM (GESTIONE TRADE E USCITE) ===
grpSweepRR = "Sweep RR System ⚙️"
enableSweepTrades = input.bool(true, "Enable Sweep RR Trades 📊", group = grpSweepRR, tooltip = "Attiva il motore di trade: ingressi sugli sweep, uscite su TP/SL/BE/Trailing.")
useCustomRR_SW = input.bool(false, "Custom RR Mode ⚙️ (ignora swing, usa RR fisso)", group = grpSweepRR, tooltip = "When enabled, TP is calculated using a fixed Risk:Reward ratio instead of the previous swing level.")
```

```
slBufferPct = input.float(0.2, "SL Buffer (% del prezzo, oltre la wick)", minval = 0.0, step = 0.05, group = grpSweepRR, tooltip = "Margine extra oltre la wick della candela per posizionare lo Stop Loss, espresso in % del prezzo d'ingresso. Es. 0.2 = 0,2% del prezzo. Vale identico su qualsiasi coin: non va mai ricalcolato. Aumentalo per ridurre gli stop presi dal rumore.")
rrRatio_SW = input.float(5.0, "Risk:Reward Ratio", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "Used in Custom RR Mode: TP = entry ± (SL distance × ratio). In standard mode the previous swing level is used as TP.")
enableBE_SW = input.bool(true, "Enable Breakeven 🔒", group = grpSweepRR, tooltip = "When price reaches the Breakeven RR level, the Stop Loss is automatically moved to entry price.")
beRatio_SW = input.float(2.0, "Breakeven at RR", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "The Risk:Reward level at which Breakeven is triggered. Example: 2.0 means BE activates when price moves 2× the SL distance in profit.")
enableTrailing_SW = input.bool(true, "Enable Trailing Stop 🔁", group = grpSweepRR)
trailStartRR_SW = input.float(3, "Trailing Stop Start at RR", minval = 0.1, step = 0.1, group = grpSweepRR, tooltip = "Defines at which Risk:Reward level the trailing activates.")
trailStepPct = input.float(0.4, "Trailing Stop Step (% del prezzo)", minval = 0.01, step = 0.05, group = grpSweepRR, tooltip = "Distanza a cui lo Stop Loss insegue il prezzo, espressa in % del prezzo d'ingresso. Es. 0.4 = 0,4% del prezzo. Vale identico su qualsiasi coin: non va mai ricalcolato.")
overlapTrades_SW = input.bool(false, "Overlap Trades 🔀", group = grpSweepRR, tooltip = "Allow multiple trades to be open simultaneously. When disabled, a new trade is only opened after the previous one closes.")

// === VISUALIZZAZIONE TRADE (opzionale) ===
grpVisual = "Trade Visual 🎨"
showTradeLevels = input.bool(true, "Show Trade Levels 📐 (linee Entry/SL/TP)", group = grpVisual, tooltip = "Disegna le linee del trade: Entry grigia tratteggiata, Stop Loss rossa, Take Profit verde. La linea SL si sposta quando scattano Breakeven o Trailing Stop. Le linee restano sul grafico dopo la chiusura per rivedere i trade passati.")
showExitMarker = input.bool(true, "Show Exit Point ● (prezzo esatto di chiusura)", group = grpVisual, tooltip = "Disegna un pallino al prezzo esatto in cui il trade si è chiuso: sulla linea verde = Take Profit, sulla linea rossa = Stop Loss, in mezzo = Trailing Stop o Breakeven.")

// === FILTRO ORARIO 1 ===
enableTimeFilter = input.bool(true, title = 'Enable Time Filter 1 🕘', group = "Time Filter 1")
timeFilter1Session = input.session("0600-1400", title = 'Time Filter 1 Range', group = "Time Filter 1", tooltip = "Session format automatically adjusts to asset timezone")
// === FILTRO ORARIO 2 ===
enableTimeFilter2 = input.bool(true, title = 'Enable Time Filter 2 🕒', group = "Time Filter 2")
timeFilter2Session = input.session("1400-2200", title = 'Time Filter 2 Range', group = "Time Filter 2", tooltip = "Session format automatically adjusts to asset timezone")
// === FILTRO ORARIO 3 ===
enableTimeFilter3 = input.bool(true, title = 'Enable Time Filter 3 🕑', group = "Time Filter 3")
timeFilter3Session = input.session("2200-0600", title = 'Time Filter 3 Range', group = "Time Filter 3", tooltip = "Session format automatically adjusts to asset timezone")

// === ALERTS SETTINGS ===
enableEntryLongAlert = input.bool(true, title = 'Enable Long Entry Alert ▲', group = "Alerts Settings 🔊")
enableEntryShortAlert = input.bool(true, title = 'Enable Short Entry Alert ▼', group = "Alerts Settings 🔊")
alertSweepTPSL = input.bool(true, "Alert Sweep TP/SL 🎯", group = "Alerts Settings 🔊", tooltip = "Send an alert when a Sweep trade hits TP, SL, Breakeven or Trailing Stop.")

// ═══════════════════════════════════════════════════════════
// FUNZIONI
// ═══════════════════════════════════════════════════════════
getTimezoneString() =>
    useUTC ? "UTC" + (utcOffset >= 0 ? "+" + str.tostring(utcOffset) : str.tostring(utcOffset)) : ""

passesVolScoreFilter() =>
    not enableVolScoreFilter or atrVolScore >= volScoreMinThreshold

vidya_calc(src_vidya, vidya_length, vidya_momentum) =>
    float momentum = ta.change(src_vidya)
    float sum_pos_momentum = math.sum(momentum >= 0 ? momentum : 0.0, vidya_momentum)
    float sum_neg_momentum = math.sum(momentum >= 0 ? 0.0 : -momentum, vidya_momentum)
    float abs_cmo = math.abs(100 * (sum_pos_momentum - sum_neg_momentum) / (sum_pos_momentum + sum_neg_momentum))
    float alpha = 2 / (vidya_length + 1)
    var float vidya_value = 0.0
    vidya_value := alpha * abs_cmo / 100 * src_vidya + (1 - alpha * abs_cmo / 100) * nz(vidya_value[1])
    ta.sma(vidya_value, 15)

// === TIME FILTER ===
isInTimeRange() =>
    if not enableTimeFilter and not enableTimeFilter2 and not enableTimeFilter3
        true
    else
        inRange1 = enableTimeFilter ? not na(time(timeframe.period, timeFilter1Session, getTimezoneString())) : false
        inRange2 = enableTimeFilter2 ? not na(time(timeframe.period, timeFilter2Session, getTimezoneString())) : false
        inRange3 = enableTimeFilter3 ? not na(time(timeframe.period, timeFilter3Session, getTimezoneString())) : false
        inRange1 or inRange2 or inRange3

// ═══════════════════════════════════════════════════════════
// TYPES
// ═══════════════════════════════════════════════════════════
type SweepTrade
    float slLevel
    float tpLevel
    float entryPrice
    float origSL
    bool isBull
    bool beTriggered
    bool trailActivated
    float trailHighWater
    int signalBar
    line entryLn
    line slLn
    line tpLn

var array<SweepTrade> activeSweepTrades = array.new<SweepTrade>()

// ═══════════════════════════════════════════════════════════
// BASE INDICATORS
```

```
// ═══════════════════════════════════════════════════════════
ema750 = ta.ema(close, 750)
smootherATR = ta.ema(ta.tr, 750)
upperBand = ema750 + atrMultiplier * smootherATR
lowerBand = ema750 - atrMultiplier * smootherATR
vwap = ta.vwap(hl2)
pipSize = syminfo.mintick * (syminfo.type == "forex" ? 10 : 1)

// VIDYA (parametri fissi come nell'originale)
vidya_length = 10
vidya_momentum = 20
vidya_value = vidya_calc(close, vidya_length, vidya_momentum)

// ═══════════════════════════════════════════════════════════
// LIQUIDITY SWEEP (logica invariata)
// ═══════════════════════════════════════════════════════════
swingHighSweepTS = ta.pivothigh(high, swingLenSweep_TS, swingLenSweep_TS)
swingLowSweepTS = ta.pivotlow(low, swingLenSweep_TS, swingLenSweep_TS)
var float lastSHSweepTS = na
var float lastSLSweepTS = na
var int lastSHBarSweepTS = na
var int lastSLBarSweepTS = na
var float prevSHSweepTS = na
var float prevSLSweepTS = na
if not na(swingHighSweepTS)
    prevSHSweepTS := lastSHSweepTS
    lastSHSweepTS := swingHighSweepTS
    lastSHBarSweepTS := bar_index - swingLenSweep_TS
if not na(swingLowSweepTS)
    prevSLSweepTS := lastSLSweepTS
    lastSLSweepTS := swingLowSweepTS
    lastSLBarSweepTS := bar_index - swingLenSweep_TS
bearSweep_TS1 = not na(lastSHSweepTS) and high > lastSHSweepTS and close < lastSHSweepTS
bullSweep_TS1 = not na(lastSLSweepTS) and low < lastSLSweepTS and close > lastSLSweepTS
bearSweep_TS2 = false
bullSweep_TS2 = false
if not na(lastSHSweepTS)
    for i = 1 to sweepLookback_TS
        if close[i] > lastSHSweepTS and close < lastSHSweepTS
            bearSweep_TS2 := true
            break
if not na(lastSLSweepTS)
    for i = 1 to sweepLookback_TS
        if close[i] < lastSLSweepTS and close > lastSLSweepTS
            bullSweep_TS2 := true
            break
bullSweep_TS = enableLiqSweep and (bullSweep_TS1 or bullSweep_TS2) and isInTimeRange() and (not sweepEMAFilter_TS or close > ema750) and (not sweepVIDYAFilter_TS or (not na(vidya_value) and close > vidya_value)) and (not sweepVWAPFilter_TS or close > vwap) and passesVolScoreFilter() and (not enableATRMode or close <= upperBand)
bearSweep_TS = enableLiqSweep and (bearSweep_TS1 or bearSweep_TS2) and isInTimeRange() and (not sweepEMAFilter_TS or close < ema750) and (not sweepVIDYAFilter_TS or (not na(vidya_value) and close < vidya_value)) and (not sweepVWAPFilter_TS or close < vwap) and passesVolScoreFilter() and (not enableATRMode or close >= lowerBand)

// ═══════════════════════════════════════════════════════════
// AGGIORNA TRADE SWEEP ATTIVI (Breakeven → TP/SL → Trailing, invariato)
// ═══════════════════════════════════════════════════════════
bool exitLongThisBar = false
bool exitShortThisBar = false
bool entryLongThisBar = false
bool entryShortThisBar = false

if array.size(activeSweepTrades) > 0
    for i = array.size(activeSweepTrades) - 1 to 0
        t = array.get(activeSweepTrades, i)
        // Breakeven
        if enableBE_SW and not t.beTriggered
            slDist = math.abs(t.entryPrice - t.origSL)
            beLevel = t.isBull ? t.entryPrice + slDist * beRatio_SW : t.entryPrice - slDist * beRatio_SW
            if (t.isBull ? high >= beLevel : low <= beLevel)
                t.slLevel := t.entryPrice
                t.beTriggered := true
        // Uscita: TP o SL (incluso SL spostato da BE/Trailing)
        slHit = t.isBull ? low <= t.slLevel : high >= t.slLevel
        tpHit = t.isBull ? high >= t.tpLevel : low <= t.tpLevel
        if slHit or tpHit
            exitPrice = tpHit ? t.tpLevel : t.slLevel
            pips = math.abs(exitPrice - t.entryPrice) / pipSize
            isProfit = t.isBull ? t.slLevel > t.entryPrice : t.slLevel < t.entryPrice
            isBE = t.slLevel == t.entryPrice
            slPipsForRR = math.abs(t.entryPrice - t.origSL) / pipSize
            tradeRR = slPipsForRR > 0 ? pips / slPipsForRR : 0.0
            if t.isBull
                exitLongThisBar := true
            else
                exitShortThisBar := true
            // Congela le linee del trade sulla barra di chiusura
            if not na(t.entryLn)
                line.set_x2(t.entryLn, bar_index)
            if not na(t.tpLn)
                line.set_x2(t.tpLn, bar_index)
            if not na(t.slLn)
                line.set_x2(t.slLn, bar_index)
                line.set_y1(t.slLn, t.slLevel)
                line.set_y2(t.slLn, t.slLevel)
```

```
            // Pallino al prezzo esatto di chiusura
            if showExitMarker
                label.new(bar_index, exitPrice, "", style = label.style_circle, color = t.isBull ? color.new(color.green, 0) : color.new(color.red, 0), size = size.tiny)
            if alertSweepTPSL
                if tpHit
                    alert("SWEEP TP HIT ✅ +" + str.tostring(pips, "#.#") + " pips / +" + str.tostring(tradeRR, "#.##") + "R", alert.freq_once_per_bar)
                else if isProfit
                    alert("SWEEP TRAIL HIT ✅ +" + str.tostring(pips, "#.#") + " pips / +" + str.tostring(tradeRR, "#.##") + "R", alert.freq_once_per_bar)
                else if isBE
                    alert("SWEEP BE HIT 🔒 0 pips", alert.freq_once_per_bar)
                else
                    alert("SWEEP SL HIT ❌ -" + str.tostring(pips, "#.#") + " pips / -" + str.tostring(tradeRR, "#.##") + "R", alert.freq_once_per_bar)
            array.remove(activeSweepTrades, i)
        else
            // Trailing Stop
            if enableTrailing_SW
                slDist = math.abs(t.entryPrice - t.origSL)
                trailTrigger = t.isBull ? t.entryPrice + slDist * trailStartRR_SW : t.entryPrice - slDist * trailStartRR_SW
                trailStep = t.entryPrice * trailStepPct / 100
                if not t.trailActivated
                    if (t.isBull ? high >= trailTrigger : low <= trailTrigger)
                        t.trailActivated := true
                        t.trailHighWater := t.isBull ? high : low
                if t.trailActivated
                    if t.isBull
                        if high > t.trailHighWater
                            t.trailHighWater := high
                        newSL = t.trailHighWater - trailStep
                        if newSL > t.slLevel
                            t.slLevel := newSL
                    else
                        if low < t.trailHighWater
                            t.trailHighWater := low
                        newSL = t.trailHighWater + trailStep
                        if newSL < t.slLevel
                            t.slLevel := newSL
            // Estendi le linee del trade aperto e aggiorna la linea SL (BE/Trailing)
            if not na(t.entryLn)
                line.set_x2(t.entryLn, bar_index)
            if not na(t.tpLn)
                line.set_x2(t.tpLn, bar_index)
            if not na(t.slLn)
                line.set_x2(t.slLn, bar_index)
                line.set_y1(t.slLn, t.slLevel)
                line.set_y2(t.slLn, t.slLevel)

// ═══════════════════════════════════════════════════════════
// APRI NUOVI TRADE SWEEP (INGRESSI, logica invariata - buffer in % del prezzo)
// ═══════════════════════════════════════════════════════════
sw_canLong = overlapTrades_SW or array.size(activeSweepTrades) == 0
sw_canShort = overlapTrades_SW or array.size(activeSweepTrades) == 0
if enableSweepTrades and bullSweep_TS and sw_canLong
    entry = close
    slDist = math.max((entry - low), pipSize) + entry * slBufferPct / 100
    tpLvl = useCustomRR_SW
         ? entry + slDist * rrRatio_SW
         : (not na(prevSHSweepTS) and prevSHSweepTS > entry ? prevSHSweepTS : entry + slDist * rrRatio_SW)
    line eLn = showTradeLevels ? line.new(bar_index, entry, bar_index + 1, entry, color = color.new(color.gray, 30), style = line.style_dashed, width = 1) : na
    line sLn = showTradeLevels ? line.new(bar_index, entry - slDist, bar_index + 1, entry - slDist, color = color.new(color.red, 0), style = line.style_dotted, width = 1) : na
    line tLn = showTradeLevels ? line.new(bar_index, tpLvl, bar_index + 1, tpLvl, color = color.new(color.green, 0), style = line.style_dotted, width = 1) : na
    array.push(activeSweepTrades, SweepTrade.new(entry - slDist, tpLvl, entry, entry - slDist, true, false, false, float(na), bar_index, eLn, sLn, tLn))
    entryLongThisBar := true
if enableSweepTrades and bearSweep_TS and sw_canShort
    entry = close
    slDist = math.max((high - entry), pipSize) + entry * slBufferPct / 100
    tpLvl = useCustomRR_SW
         ? entry - slDist * rrRatio_SW
         : (not na(prevSLSweepTS) and prevSLSweepTS < entry ? prevSLSweepTS : entry - slDist * rrRatio_SW)
    line eLn = showTradeLevels ? line.new(bar_index, entry, bar_index + 1, entry, color = color.new(color.gray, 30), style = line.style_dashed, width = 1) : na
    line sLn = showTradeLevels ? line.new(bar_index, entry + slDist, bar_index + 1, entry + slDist, color = color.new(color.red, 0), style = line.style_dotted, width = 1) : na
    line tLn = showTradeLevels ? line.new(bar_index, tpLvl, bar_index + 1, tpLvl, color = color.new(color.green, 0), style = line.style_dotted, width = 1) : na
    array.push(activeSweepTrades, SweepTrade.new(entry + slDist, tpLvl, entry, entry + slDist, false, false, false, float(na), bar_index, eLn, sLn, tLn))
    entryShortThisBar := true

// ═══════════════════════════════════════════════════════════
// VISUALIZZAZIONE - I 4 PUNTI RICHIESTI
// ═══════════════════════════════════════════════════════════
plotshape(entryLongThisBar, title = 'Ingresso Long ▲', location = location.belowbar, color = color.new(color.green, 0), style = shape.triangleup, size = size.small)
plotshape(entryShortThisBar, title = 'Ingresso Short ▼', location = location.abovebar, color = color.new(color.red, 0), style = shape.triangledown, size = size.small)
plotshape(exitLongThisBar, title = 'Uscita Long ×', location = location.abovebar, color = color.new(color.green, 0), style = shape.xcross, size =
```

```
size.small)
 316  plotshape(exitShortThisBar, title = 'Uscita Short ×', location = location.belowbar, color = color.new(color.red, 0), style = shape.xcross, size =
size.small)
 317
 318  // ═══════════════════════════════════════════════════════
 319  // ALERTS
 320  // ═══════════════════════════════════════════════════════
 321  alertcondition(entryLongThisBar, title = 'Ingresso Long ▲', message = '▲ LONG ENTRY (Bullish Liquidity Sweep)')
 322  alertcondition(entryShortThisBar, title = 'Ingresso Short ▼', message = '▼ SHORT ENTRY (Bearish Liquidity Sweep)')
 323  alertcondition(exitLongThisBar, title = 'Uscita Long ×', message = '× LONG EXIT (TP/SL/BE/Trailing)')
 324  alertcondition(exitShortThisBar, title = 'Uscita Short ×', message = '× SHORT EXIT (TP/SL/BE/Trailing)')
 325  if entryLongThisBar and enableEntryLongAlert
 326      alert('▲ LONG ENTRY (Bullish Liquidity Sweep) at ' + str.tostring(close, format.mintick), alert.freq_once_per_bar_close)
 327  if entryShortThisBar and enableEntryShortAlert
 328      alert('▼ SHORT ENTRY (Bearish Liquidity Sweep) at ' + str.tostring(close, format.mintick), alert.freq_once_per_bar_close)
 329
```